

Booking Backend

Database Schema Reference — Tables, Relationships & Example Data

Shared PostgreSQL database for:
Centurion Hotel & Mum's Garden Resort

Node.js + Express · Sequelize ORM · one DB, `property_id`-scoped | July 2026

Contents

1	How the schema is organized
2	Entity relationship overview
3	Table reference — definition + example rows for all 18 tables
4	Availability logic (computed, not stored)
5	Background jobs that touch this schema
6	Fields still pending client sign-off

1. How the schema is organized

One database, one set of tables, both properties. Centurion Hotel and Mum's Garden Resort are not separate schemas or separate databases — every tenant-scoped table carries a `property_id` foreign key, and a Sequelize default scope (`resolveProperty` middleware) guarantees a query for one property can never leak rows from the other.

Property	Slug	Example <code>id</code> (UUID)
Centurion Hotel CENTURION	<code>centurion-hotel</code>	<code>b3ac1eab-5b2d-4e2a-9c1a-111111111111</code>
Mum's Garden Resort MUMS GARDEN	<code>mums-garden-resort</code>	<code>d7f2c9a4-8e31-4b7a-9f2e-222222222222</code>

Two design choices worth knowing before you read the table-by-table reference:

- **Property-specific fields live in JSONB, not fixed columns.** Centurion and Mum's Garden have different amenity sets, different room naming, different content needs. Rather than adding per-property columns, anything free-form sits in a JSONB `attributes` (or similar) column. Core bookable fields used in filtering/sorting (price, occupancy, bed count) stay as real typed columns.
- **Availability is computed, never stored.** There is no `availability` table — it's derived at query time from `room_types.total_units` minus overlapping bookings. See Section 4.

Every model file lives under `/models` with one migration per table (in dependency order) and seeders for the two `properties` rows plus default cancellation policies. Full Sequelize file layout is in the original spec if you need to regenerate migrations.

2. Entity relationship overview

```
properties
|->< room_types >>< room_type_images
|->< rooms (physical, belongs to a room_type)
|->< rate_plans >>< cancellation_policies
|->< bookings >>< booking_rooms --> rooms
|   |->< payments >>< refunds
|->< offers
|->< gallery_items
|->< inquiries      (events / group / general contact form submissions)
|->< staff_users    (admin/manager per property -- modeled as users.role)

users (guests, cross-property, account optional)
|->< bookings      (nullable FK -- guest checkout allowed)

notifications_log  (generic, polymorphic on booking_id)
audit_log          (generic, polymorphic on entity_type/entity_id)
media_assets       (shared upload registry -- feeds gallery/room-images/offers/event-types)
```

3. Table reference

Every table below shows its column definition followed by 2–3 realistic example rows, using the two seeded properties. Foreign keys are tagged **FK** , primary keys **PK** .

3.1 properties

The tenant table. Every other tenant-scoped table hangs off this via `property_id` .

Column	Type	Notes
PK id	UUID	
slug	STRING, unique	<code>centurion-hotel</code> , <code>mums-garden-resort</code>
name	STRING	
timezone	STRING	default <code>Africa/Nairobi</code>
currency	STRING(3)	default <code>KES</code>
contact_email / contact_phone / address	STRING / STRING / TEXT	
check_in_time / check_out_time	TIME	e.g. <code>14:00</code> / <code>10:00</code> — single source of truth
is_active	BOOLEAN	
created_at / updated_at	TIMESTAMP	

EXAMPLE ROWS

id	slug	name	currency	check_in_time	check_out_time
b3ac1eab...1111	centurion-hotel	Centurion Hotel	KES	14:00	10:00
d7f2c9a4...2222	mums-garden-resort	Mum's Garden Resort	KES	14:00	10:00

3.2 room_types

Unique constraint on `(property_id, slug)` .

Column	Type	Notes
PK id	UUID	
FK property_id	UUID → properties	
slug / name / description	STRING / STRING / TEXT	
size_sqm	DECIMAL	typed — used for display/sort
bed_type	ENUM	<code>single</code> , <code>twin</code> , <code>double</code> , <code>queen</code> , <code>king</code> , <code>bunk</code>
max_adults / max_children	INTEGER	
total_units	INTEGER	how many physical rooms of this type exist
base_price	DECIMAL(10,2)	per-night, before rate plan overrides
currency	STRING(3)	
attributes	JSONB	free-form per-property specs, e.g. amenity tags
is_active	BOOLEAN	

EXAMPLE ROWS

id	property_id	slug	name	bed_type	total_units	base_price	attributes
a1111111...0001	...2222 (Mum's)	deluxe-garden-room	Deluxe Garden Room	queen	8	8500.00	{ "amenities": ["Garden View", "Rain Shower", "WiFi"] }
a1111111...0002	...2222 (Mum's)	standard-room	Standard Room	double	12	5500.00	{ "amenities": ["Garden View", "WiFi"] }
c2222222...0001	...1111 (Centurion)	city-view-suite	City View Suite	king	6	12000.00	{ "amenities": ["City View", "Rain Shower", "TV+VOD"] }

3.3 room_type_images

Column	Type	Notes
PK id	UUID	
FK room_type_id	UUID → room_types	
url / alt_text	STRING / STRING	
sort_order	INTEGER	
is_cover	BOOLEAN	

EXAMPLE ROWS

id	room_type_id	url	sort_order	is_cover
img-001	a1111111...0001	https://cdn.../deluxe-1.jpg	0	true
img-002	a1111111...0001	https://cdn.../deluxe-2.jpg	1	false

3.4 rooms (physical inventory)

Used for admin room assignment and maintenance blocking. Booking availability math (Section 4) is driven by `room_types.total_units` minus overlapping bookings, not this table — so front-desk can assign a physical room without it being a hard dependency for the booking engine.

Column	Type	Notes
PK id	UUID	
FK property_id	UUID → properties	
FK room_type_id	UUID → room_types	
room_number / floor	STRING / STRING	
status	ENUM	<code>active, maintenance, out_of_service</code>

EXAMPLE ROWS

id	room_type_id	room_number	floor	status
room-101	a1111111...0001	101	1	active
room-102	a1111111...0001	102	1	maintenance

3.5 cancellation_policies

Column	Type	Notes
PK id	UUID	
FK property_id	UUID → properties	
name	STRING	
tiers	JSONB	array of refund tiers, see example
deposit_pct	DECIMAL	e.g. 30 (%) — currently blank in source content, must be set per property before launch
is_default	BOOLEAN	

EXAMPLE ROWS

id	property_id	name	tiers	deposit_pct	is_default
cp-001	...2222 (Mum's)	Standard	[{"days_before":15,"refund_pct":100},{ "days_before":14,"refund_pct":50}, {"hours_before":48,"refund_pct":0}]	—pending—	true

3.6 `rate_plans`

Optional pricing/cancellation override layer, scoped by date range. If no active rate plan covers the requested dates, booking falls back to `room_types.base_price` and the `cancellation_policies` row with `is_default = true`.

Column	Type	Notes
<code>PK</code> id	UUID	
<code>FK</code> property_id / room_type_id	UUID	
name	STRING	
price_override	DECIMAL	
<code>FK</code> cancellation_policy_id	UUID	
valid_from / valid_to	DATE	
is_active	BOOLEAN	

EXAMPLE ROWS

id	room_type_id	name	price_override	valid_from	valid_to
rp-001	a1111111...0001	Midweek Discount	7000.00	2026-08-01	2026-08-31

3.7 `users` (guest accounts — optional, cross-property)

Column	Type	Notes
<code>PK</code> id	UUID	
email	STRING, unique	
phone	STRING	
password_hash	STRING, nullable	null for guest-checkout-only accounts
first_name / last_name	STRING	
role	ENUM	<code>guest</code> , <code>staff</code> , <code>admin</code> , <code>superadmin</code>
<code>FK</code> property_id	UUID, nullable	set only for staff/admin; null for guest and superadmin
email_verified_at	TIMESTAMP	

`role=guest` rows always have `property_id=NULL` (guests aren't scoped to one property). `role=staff/admin` rows require `property_id`. `superadmin` has `property_id=NULL` and bypasses scoping entirely.

EXAMPLE ROWS

id	email	role	property_id
usr-001	lawrence@example.com	guest	NULL
usr-002	manager@mumsgardenresort.co.ke	admin	...2222 (Mum's)
usr-003	ops@straightgroup.co.ke	superadmin	NULL

3.8 bookings

Column	Type	Notes
PK id	UUID	
booking_code	STRING(10), unique	human-readable, e.g. <code>MG-4F82A1</code>
FK property_id / room_type_id / rate_plan_id	UUID	rate_plan_id nullable
FK user_id	UUID, nullable	null = guest checkout
guest_first_name, guest_last_name, guest_email, guest_phone	STRING	always captured even if user_id set
check_in_date / check_out_date	DATE	
num_adults / num_children / num_rooms	INTEGER	num_rooms default 1
special_requests	TEXT	
status	ENUM	<code>pending_payment, confirmed, checked_in, checked_out, cancelled, no_show, expired</code>
subtotal / deposit_required / amount_paid / balance_due	DECIMAL(10,2)	balance_due kept as a column for fast admin queries
cancellation_deadline_at	TIMESTAMP	computed from policy tiers
source	ENUM	<code>web, whatsapp, admin_manual, phone</code>
expires_at	TIMESTAMP	pending_payment bookings auto-expire to release inventory

EXAMPLE ROWS

booking_code	property_id	room_type_id	check_in / out	status	subtotal	amount_paid
MG-4F82A1	...2222 (Mum's)	a1111111...0001	2026-08-10 / 2026-08-12	pending_payment	17000.00	0.00
CH-9B21D4	...1111 (Centurion)	c2222222...0001	2026-09-02 / 2026-09-04	confirmed	24000.00	7200.00

3.9 booking_rooms (join table)

Supports multi-room bookings and post-hoc physical room assignment.

Column	Type	Notes
PK id	UUID	
FK booking_id	UUID → bookings	
FK room_id	UUID → rooms, nullable	assigned at/near check-in
FK room_type_id	UUID → room_types	
sequence	INTEGER	

EXAMPLE ROWS

id	booking_id	room_id	room_type_id	sequence
br-001	MG-4F82A1	NULL	a1111111...0001	1

3.10 payments

Column	Type	Notes
PK id	UUID	
FK booking_id	UUID → bookings	
provider	ENUM	mpesa, cash, card (card reserved, not wired in v1)
type	ENUM	deposit, balance, full
amount / currency	DECIMAL(10,2) / STRING(3)	
status	ENUM	initiated, pending, completed, failed, reversed
provider_reference	STRING	M-Pesa CheckoutRequestID
mpesa_receipt_number	STRING, nullable	populated from callback on success
phone_number	STRING	the paying MSISDN
raw_callback	JSONB	full Daraja callback payload, stored for audit/debugging
initiated_at / completed_at	TIMESTAMP	

EXAMPLE ROWS

id	booking_id	type	amount	status	mpesa_receipt_number
pay-001	MG-4F82A1	deposit	5100.00	pending	NULL
pay-002	CH-9B21D4	deposit	7200.00	completed	QK7T8X9Y1Z

3.11 refunds

Column	Type	Notes
PK id	UUID	
FK booking_id / payment_id	UUID	
amount / reason	DECIMAL / TEXT	
status	ENUM	requested, processing, completed, rejected
FK processed_by	UUID → users	

EXAMPLE ROWS

id	booking_id	amount	status	processed_by
ref-001	CH-9B21D4	3600.00	requested	NULL

3.12 offers

Column	Type	Notes
PK id	UUID	
FK property_id	UUID	
title / description	STRING / TEXT	
discount_type	ENUM	percentage, fixed_amount
discount_value	DECIMAL	
valid_from / valid_to	DATE	
image_url / terms	STRING / TEXT	
is_active	BOOLEAN	

EXAMPLE ROWS

id	property_id	title	discount_type	discount_value
off-001	...2222 (Mum's)	Midweek Escape	percentage	20

3.13 gallery_items

Column	Type	Notes
PK id	UUID	
FK property_id	UUID	
category	ENUM	rooms, dining, events, exterior
image_url / caption	STRING / STRING	
sort_order	INTEGER	

EXAMPLE ROWS

id	property_id	category	caption	sort_order
gal-001	...2222 (Mum's)	dining	Sunset dinner on the terrace	2

3.14 inquiries

Non-booking contact / events / group form submissions.

Column	Type	Notes
PK id	UUID	
FK property_id	UUID	
type	ENUM	general, group_booking, events, meetings
name / email / phone / message	STRING / STRING / STRING / TEXT	
party_size	INTEGER, nullable	
preferred_date	DATE, nullable	
status	ENUM	new, contacted, closed

EXAMPLE ROWS

id	property_id	type	name	party_size	status
inq-001	...2222 (Mum's)	group_booking	Jane W.	30	new

3.15 notifications_log

Generic, polymorphic on booking_id . The whatsapp channel value and this table's shape exist now specifically so the future n8n workflow can write/read against the same log without a schema change.

Column	Type	Notes
PK id	UUID	
FK booking_id	UUID	
channel	ENUM	email, sms, whatsapp
template	STRING	
status	ENUM	queued, sent, failed
sent_at	TIMESTAMP	
payload	JSONB	

EXAMPLE ROWS

id	booking_id	channel	template	status
notif-001	MG-4F82A1	email	booking_confirmation	sent

3.16 audit_log

Column	Type	Notes
PK id	UUID	
FK actor_user_id	UUID, nullable	
action	STRING	
entity_type / entity_id	STRING / UUID	polymorphic
changes	JSONB	

EXAMPLE ROWS

id	actor_user_id	action	entity_type	entity_id
aud-001	usr-002	room_type.update	room_type	a1111111...0001

3.17 event_types (Meetings & Events content cards)

Content cards, not scheduled/dated events — private dinners, cocktail parties, baby/bridal showers, birthdays, corporate meetings, team training, etc. Admin can add/edit/reorder/retire without a code deploy. Actual enquiries against these come through `inquiries` (3.14), not a new booking flow.

Column	Type	Notes
PK id	UUID	
FK property_id	UUID	
title / description	STRING / TEXT	
image_url / icon	STRING / STRING, nullable	
sort_order	INTEGER	
is_active	BOOLEAN	

EXAMPLE ROWS

id	property_id	title	icon	sort_order
evt-001	...1111 (Centurion)	Corporate Board Meetings	briefcase	1
evt-002	...2222 (Mum's)	Bridal Showers	flower	3

3.18 media_assets (shared upload registry)

A single upload endpoint writes here and returns a URL; `gallery_items`, `room_type_images`, `offers.image_url`, and `event_types.image_url` all just store the resulting URL — file-handling logic lives in one place instead of duplicated per content type.

Column	Type	Notes
PK id	UUID	
FK property_id	UUID	
url / original_filename / mime_type	STRING	
size_bytes	INTEGER	
FK uploaded_by	UUID → users	

EXAMPLE ROWS

id	property_id	original_filename	mime_type	size_bytes
media-001	...2222 (Mum's)	deluxe-1.jpg	image/jpeg	842011

4. Availability logic (computed, not stored)

There is no `availability` table. Available units for a room type over a date range is computed on every request:

```
available_units = room_types.total_units
- COUNT(bookings WHERE room_type_id = X
      AND status IN ('confirmed','pending_payment','checked_in')
      AND check_in_date < :checkOut AND check_out_date > :checkIn)
```

`pending_payment` bookings count against availability until `expires_at` passes — this is what stops two guests both getting an M-Pesa prompt for the last room.

5. Background jobs that touch this schema

Job	Frequency	What it touches
Expire stale bookings	every 1-2 min	flips <code>bookings.status</code> from <code>pending_payment</code> → <code>expired</code> once past <code>expires_at</code> , freeing inventory
Daraja STK timeout handling	on timeout	marks <code>payments.status = 'failed'</code> if no callback arrives in Daraja's window
Booking confirmation notifications	on <code>confirmed</code> status	enqueues a row in <code>notifications_log</code> (email, +SMS if configured)
Cancellation deadline reminder (optional)	daily	notifies guests approaching <code>cancellation_deadline_at</code>

A simple `node-cron` in-process job is enough at current volume — move to a real queue only if it's needed later.

6. Fields still pending client sign-off

These affect the actual data going into rows, not the schema shape — don't seed placeholder values for any of these:

- Confirmed room specs (size / bed type / occupancy / price) per room type, per property.
- One official `check_in_time` / `check_out_time` per property.
- `cancellation_policies.deposit_pct` — currently blank in source content.
- Whether card payments get added later (schema already reserves `payments.provider = 'card'` , no code path yet).